COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010-Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	08-18-2026	**DATE SUBMITTED:**	08-09-2026
NAME:	DOLOSO JR, ANDREW S.	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

Hands-on Activity 6.1 Searching Techniques

1. PROCEDURE OUTPUT

main.cpp

```
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "node.h"
#include "searchingAlgo.h"

const int max_size = 50;

int main() {
    //generate random varable
    int dataset[max_size];
    srand(time(0));
    for(int i=0; i<max_size; i++){
        dataset[i]=rand();

    }
    for(int i=0; i < max_size; i++) {
        std::cout<<dataset[i]<< "";
    }

     std::cout<< std::endl;
     //create a user imput to ask what value to find:
     int toFind;
     std::cout<< "what value you want to search";
     std::cin>> toFind;
```

```
    linearSearchArr(dataset, max_size, toFind);

    //linear search:

    //linked list:
    Node<char>* name1 = new_node('J');
    Node<char>* name2 = new_node('I');
    Node<char>* name3 = new_node('M');
    Node<char>* name4 = new_node('L');
    Node<char>* name5 = new_node('O');
    Node<char>* name6 = new_node('R');
    Node<char>* name7 = new_node('D');

    name1->next = name2;
    name2->next = name3;
    name3->next = name4;
    name4->next = name5;
    name5->next = name6;
    name6->next = name7;

    std::cout<<"Linear search for Linked List result: ";
    linearSearchLL(name1, 'D');
    std::cout<<"Linear search for Linked List result: ";
    linearSearchLL(name1, 'x');

    //implement here:
}
```

searchingArr.h

```
#include "node.h"

#ifndef SEARCHING_ALGO_H
#define SEARCHING_ALGO_H


template <typename T>
```

```cpp
void linearSearchArr(T data[], int arrSize, T dataFind){
    //Step 1: I := 0
    int i = 0;

    //Repeat while I <= n
    while( i < arrSize){
        if(dataFind == data[i]){
            std::cout<<"the search is success\n";
            return;
        }
        //increment i to move to the next element
        i++;
    }
    std::cout<< "the element is not found \n";

    return;
}

template <typename T>
void linearSearchLL(Node<T>* head, T dataFind){
   //repeat the loop until the current node is not null
   while(head != nullptr){

    // if item = current node's data then display searching is
successful
    if(dataFind == head->data){
        std::cout<<"the search was a success\n";
        return;
    }

    // if not, move to the next node
    head = head->next;
}

    // the searching reached nullptr, -> search is unseccessful
    std::cout<< "the search was unsuccessful\n";

}
```

```
/*
1. Start node is set to head of the list and the last node is set to
NULL.
2. Middle element is calculated using the two pointers approach
discussed above.
3. If the middle element is same as the key to be searched, we return
it.
4. Else if middle element is greater than the key to be searched, we
have to search is the
right side of the singly linked list. So, we set start pointer to the
next of middle
element.
5. Else if middle element is less than the key to be searched, we have
to search is the left
side of the singly linked list. So, we set last pointer to the middle
element.
6. If the key is found or the entire linked list gets traversed, we
break the loop.

*/

template <typename T>
Node<T>* getMiddle(Node<T>* start, Node<T> last){
    //use two pointers: 1 slow moves one node at a time, fast moves two
nodes at a time
    Node<T>* slow = start;
    Node<T>* fast = start;

    //get middle:
    while(fast != last && fast->next != last){
        //move slow (one step)
        slow = slow->next;
        //moove fast (two steps)
        fast = fast->next->next;
    }

    // when fast reaches the end
```

```
    // slow poitns to the middle node

    return slow;
}



#endif
```

node.h

```
#ifndef NODE_H
#define NODE_H

template <typename T>
class Node{
    public:
    T data;
    Node *next;
};

template <typename  T>
Node<T> *new_node(T newData){
    Node<T> *newNode = new Node<T>;
    newNode->data = newData;
    newNode->next = NULL;
    return newNode;
}

#endif
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

COMPUTER ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: *Grammarly*

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: *grammar correction*

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.